Sierpinski Carpet

Jana Stanisavoska
UP FAMNIT
89221066
E-mail: stanisavoskajana@gmail.com

ABSTRACT

In this report, we explore the computation of the Sierpinski Carpet fractal through sequential, parallel, and distributed methods. We begin by detailing a sequential approach where the fractal is generated using a single-threaded process, followed by a parallel approach that enhances efficiency by employing Java's Fork/Join framework. This approach divides the computation into smaller tasks processed concurrently across multiple threads. The distributed approach further scales the computation by distributing tasks across multiple processors, making it capable of handling larger fractals. Each method is measured for its performance and scalability, providing insights into its strengths and trade-offs in handling the recursive complexity of the Sierpinski Carpet. The report concludes with an overview of a graphical user interface (GUI) that visualizes the fractal patterns computed by these methods, offering interactive features.

1 INTRODUCTION

The Sierpinski Carpet is a well-known fractal that reveals an intriguing iterative approach to geometric construction.

To generate the Sierpinski Carpet, we start with a solid-filled square, denoted as $C(0)$. This square is subdivided into nine smaller, congruent squares, and the centre square is removed to create $C(1)$. This process of subdividing and extracting the central square is repeated for each of the remaining eight squares, resulting in $C(2)$, $C(3)$, and so forth. The resulting fractal C, is the intersection of all these iterative sets, defined as

$$C = \bigcap_{n=0}^{\infty} C(n)$$

Each iteration filters the fractal by removing the central square of every remaining smaller square, leading to an increasingly complex and detailed pattern. The process scales each square by a factor of $r = 1/3$ and is repeated infinitely, resulting in a fractal structure with an infinitely detailed boundary.

To compute the Sierpinski Carpet, one can employ a sequential algorithm that systematically applies recursive logic to generate each fractal level.

The sequential approach involves iterating over each square, subdividing it, and determining which sub-squares remain and which are removed according to the Sierpinski Carpet construction rules. This method directly calculates the fractal, using a single-threaded process to iterate through each level of depth for constructing the carpet.

Although the sequential method is simple and easy to implement, it is computationally intensive as the number of iterations increases. Each iteration exponentially increases the number of calculations required, making it challenging to compute deeper fractal levels due to time and memory constraints.

Thus, the rest of the report will discuss the computational methods used to generate, visualize it and compare their performance including sequential, parallel, and distributed approaches.

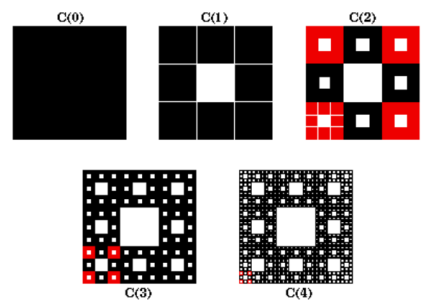

Figure 1: An illustration of the first few iterations in the construction of the Sierpinski Carpet.

2 DESIGN

To efficiently compute the Sierpinski Carpet, a parallelized approach is employed to distribute the workload across multiple computational units, enhancing both performance and scalability. The design of this approach centres on partitioning the overall computation into smaller, manageable tasks that can be processed concurrently. This is accomplished by dividing the fractal's grid into sections that can be computed independently, allowing multiple processors or threads to work simultaneously without interference.

In our project, each worker—whether a thread in a multi-threaded environment or a process in a distributed system—is assigned a specific grid region. The fractal's recursive nature allows these regions to be processed in isolation one from another, as each sub-region can independently apply the Sierpinski Carpet's recursive removal of central squares. By dividing the grid into smaller blocks, each worker can compute its assigned section and store only the relevant data, minimizing memory usage and maximizing cache efficiency.

However, a fixed assignment of grid regions to workers may lead to an imbalance in workload distribution, as some areas of the fractal may require more computations than others, depending on the recursion depth and the pattern's complexity. To address this, a dynamic task distribution strategy can be implemented, large amount of small tasks are generated and assigned to workers on-demand, rather than at the beginning. This approach helps balance the workload proportionally across all workers, ensuring that all computational resources are utilized effectively and minimizing idle time.

The parallel approach in our design also considers the overhead associated with communication and synchronization among workers. By carefully choosing the size of the tasks, we ensure that the cost of managing these tasks does not outweigh the benefits of parallelism. This is particularly important in distributed systems, where communication latency between nodes can become a bottleneck if not managed properly.

Both in parallel and distributed environments, our design leverages a job queue mechanism where tasks are dynamically allocated to workers. This design not only maximizes the use of available computational resources but also provides flexibility in scaling the computation across different hardware configurations, from a single multi-core machine to a distributed cluster of nodes. After computation, the results from each worker are gathered and combined to form the final representation of the Sierpinski Carpet.

Overall, this parallel design enhances computational efficiency and scalability, providing a robust framework for generating the Sierpinski Carpet at varying depths and resolutions. Subsequent sections will further detail the specific implementations for parallel and distributed environments, comparing their performance and discussing potential optimizations.

3 PROBLEM DESCRIPTION

The project involves generating and visualizing the Sierpiński Carpet fractal through multiple execution modes. Each mode is designed to tackle the challenge of rendering this fractal efficiently and effectively based on the available computing resources. The project is divided into three main components: sequential, parallel, and distributed computation, each utilizing different strategies to handle the fractal generation.

Sequential Computation

In sequential mode, the fractal generation is performed using a single thread. This approach is the simplest form and serves as the baseline for more complex methods.

The process is prompted by reading the specified recursion depth, which dictates the level of detail and complexity of the fractal. The program then calculates the grid size required for rendering the fractal and initializes a data array to hold the fractal information. The fractal computation is accomplished by recursively subdividing the grid and filling in the appropriate cells based on the Sierpiński Carpet's pattern. After the fractal is computed, the results are written to a file for visualization. This mode is useful for validating correctness and understanding the basic behaviour of the fractal generation algorithm.

Parallel Computation

The parallel mode extends the sequential approach by leveraging multi-threading to enhance performance during the Sierpiński Carpet generation. In this mode, the ForkJoin framework efficiently distributes the fractal computation across multiple threads. The process begins by setting the grid size and recursion depth, which are determined based on user input or default values. The ForkJoinPool, a key component of this framework, manages a pool of worker threads, each responsible for executing a 'ComputeTask'.

The 'ComputeTask' class extends 'RecursiveAction', making it a recursive task suited for parallel execution. Within the 'compute()' method, the task checks the depth of recursion to determine if further division is necessary. If a further division is possible, the task creates new sub-tasks (up to 8 smaller sub-tasks, excluding the central block) and invokes them using 'invokeAll(tasks)'. This recursive subdivision allows the task to be split into smaller pieces, enabling parallel processing of different sections of the grid and optimizing performance. This approach recursively breaks down the fractal computation into subtasks until they are small enough to be executed independently by a thread, embodying the "divide and conquer" principle. This ensures that each thread processes a manageable section of the grid, skipping the central block to maintain the Sierpiński Carpet pattern.

A crucial aspect of this parallelization is the work-stealing mechanism. If certain threads complete their tasks faster than others, they can "steal" tasks from the queues of busier threads. This dynamic load balancing improves the overall efficiency and utilization of available threads, preventing any thread from remaining idle.

```
private static class ComputeTask extends RecursiveAction {
    ...
    @Override
    protected void compute() {
        if (depth == 0) { // Base case check
            return; // Stop recursion
        }
        ...
        invokeAll(tasks); // Split and execute subtasks
    }
}
```

Distributed Computation

In the MPI-based distributed mode of the Sierpiński Carpet project, the program initializes MPI to leverage parallel processing for fractal computation. After setting up the grid size and recursion depth, the root process (rank 0) broadcasts these parameters to all participating processes. Each process, identified by its rank, is allocated a specific range of rows to compute, effectively partitioning the fractal's data space.

The computeFractal method is then executed in each process, where it computes its assigned portion of the fractal and stores the results in a local data array. MPI's communication functions are used to coordinate and consolidate the results. Once all processes complete their calculations, they send their local data to the root process, which gathers and merges these partial results into a complete fractal dataset. The root process writes the collected data to an output file. This approach effectively divides the workload across multiple processes, leveraging MPI for parallel computation and communication, and ensures the efficient generation of the Sierpinski Carpet fractal at a potentially large scale.

```
// Calculate number of rows each process is responsible for
// Distribute rows
int rowsPerProcess = (gridSize + size - 1) / size;

// Each process starts from a different row
int startRow = rank * rowsPerProcess;

// Calculate end row
int endRow = Math.min(startRow + rowsPerProcess, gridSize);
```

4 KEY COMPONENTS ACROSS EXECUTION MODES

Despite the differences in execution modes, several fundamental components are integral to all modes. Understanding these components provides insight into the fractal generation process as a whole. The key components shared across all execution modes are:

Initialization and Parameter Setup

The Sierpiński Carpet fractal generation process begins with the initialization of critical parameters, notably the recursion depth and grid size. The recursion depth determines the complexity of the fractal and is the primary factor in computing the grid size.

Recursion Depth: Recursion depth (d) determines the number of levels of recursion applied in generating the fractal. Each level further subdivides the grid, affecting the final detail of the fractal pattern. A default recursion depth is set, this value can be overridden by providing a command-line argument that is parsed and validated. If parsing fails or the provided depth is invalid, the default value is used.

Grid Size Calculation: The grid size is computed based on the recursion depth using the formula 3^d, where d is the recursion depth. This formula arises from the nature of the Sierpiński Carpet fractal, which involves recursive subdivisions into a 3 x 3 grid. For instance, with a recursion depth of 4, the grid size is calculated as $3^4 = 81$. This results in a grid with 81 x 81 cells, where each cell represents a pixel in the final image of the fractal.

```
//calculate grid size
int gridSize = (int) Math.pow (3, recursionDepth);

//store the fractal data
int[] data = new int [gridSize * gridSize];
```

Initialization: The fractal information is stored in a data array, which serves as the repository for the pixel values representing the Sierpiński Carpet. This data array is a fundamental component in all execution modes. In sequential computation, this array is managed by a single thread.

The correct management and updating of this array are crucial for ensuring that the fractal is accurately represented. Each cell in the array corresponds to a pixel in the final visualization of the fractal, and thus, the array must be correctly populated based on the recursive subdivision logic.

Recursive Subdivision

Fractal Computation: The Sierpiński Carpet generation relies on a recursive subdivision approach to create its fractal pattern. This method involves breaking down the grid into smaller blocks and applying the fractal's rules to each block. Specifically, the grid is divided into nine equal-sized sub-blocks, with the central block being omitted in each subdivision. This process continues recursively until the smallest unit of the grid is reached, defined by the base case of the recursion.

In the 'computeFractal' function, the algorithm starts by checking if the recursion depth has reached zero. If it has, the function returns, marking the end of the recursive process for this path. Otherwise, it computes the new size for the sub-blocks by dividing the current size by three. The function then iterates through each row and column of the grid. For each cell '(x, y)', it determines whether the cell is part of the fractal pattern using the 'isInFractal' function. If the cell is within the not fractal, it is set to 1; otherwise, it is set to 0.

Algorithm 1 Compute Fractal

```
function computeFractal(data, x, y, size, depth)
    if depth = 0 then
        return                                  ▷ base case
    end if
    newSize ← size / 3
    for each row from startRow to endRow do
        for each column in the grid do
            if isInFractal (x, y) then
                data[x, y] ← 1
            else
                data[x, y] ← 0
            end if
        end for
    end for
    if newSize > 0 then
        for i = 0 to 2 do
            for j = 0 to 2 do
                if i = 1 and j = 1 then
                    continue                    ▷ center block empty
                end if
                computeFractal(data, x + i * newSize, y + j *
newSize, newSize, depth - 1)
            end for
        end for
    end if
end function
```

The 'isInFractal' function helps decide if a cell '(x, y)' should be included in the fractal pattern. It operates by checking if the coordinates fall into the central empty block of any subdivision level. The function iteratively updates the coordinates and size to check smaller and smaller sub-blocks. If the cell is in the central block of any level, it is excluded from the fractal. If not, the function continues to refine its check until the size is reduced to zero.

Algorithm 2 Check if (x, y) is in the fractal

```
function isInFractal(x, y, size)
    while size > 0 do
        if (x % 3 = 1) and (y % 3 = 1) then
            return false                        ▷ inside center block
        end if
        x ← x / 3
        y ← y / 3
        size ← size / 3
    end while
    return true
end function
```

This recursive approach remains consistent across different computing modes—sequential, parallel, and distributed. In each mode,

the core principle of recursively subdividing the grid and applying the fractal rules ensures that the final pattern maintains the self-similar structure characteristic of the Sierpiński Carpet. Whether executed sequentially on a single processor or distributed across multiple processors or nodes, the recursive steps generate the fractal pattern by building up from the base case through the specified subdivisions.

File Output

Visualization: After completing the fractal computation, the results are written to an output file. This file contains the final data of the fractal, which is used for visualization. This file contains the final fractal data, which can be employed to generate graphical representations of the Sierpiński Carpet, providing a concrete view of the fractal's complex pattern. The file output serves as the bridge between computation and visualization, making the fractal accessible for further use. The file output process is uniform across all execution modes, ensuring that regardless of the method used (sequential, parallel, or distributed), the final fractal data is consistently formatted and stored for subsequent analysis or display.

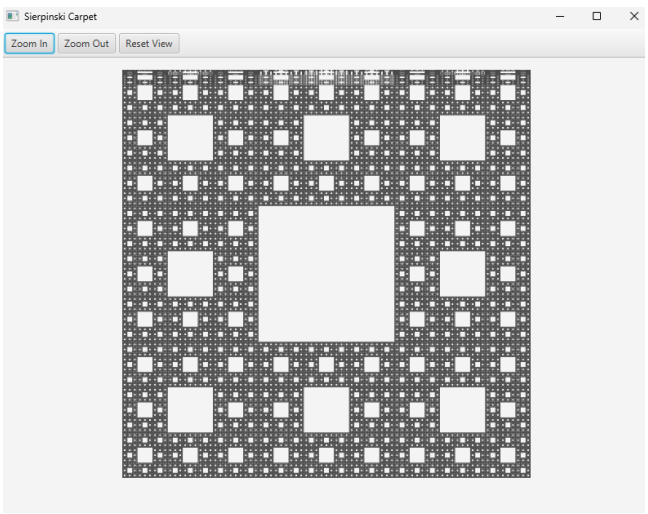

Figure 2: Output from the JavaFX application for depth 6 computed in a distributive way

Once the fractal data is computed and saved to the output file, the JavaFX-based graphical interface is used to visualize this data. The visualization process involves reading the fractal data from the file and rendering it on a canvas within a graphical window. This is achieved by first determining the grid size from the file, which is used to set the appropriate scale and offsets for displaying the fractal. The canvas is scaled and translated to fit the fractal into the display area, and then the fractal is drawn by iterating over the data and filling rectangles based on the values read, where each 1 in the data indicates a filled cell.

The graphical interface also includes interactive features such as zooming and panning. Users can zoom in and out or reset the view using buttons on a toolbar, and they can pan the view by dragging the mouse. These controls are managed through a ViewController class that handles transformations and user interactions, ensuring a responsive and adjustable visualization experience.

5 TECHNICAL REMARKS

Our project leverages JavaFX for its graphical user interface, enabling an interactive and responsive visualization of the Sierpiński Carpet. JavaFX provides a scalable canvas and user controls for zooming and panning, enhancing the visual experience.

For computational tasks, we use the ForkJoin framework to parallelize fractal generation. The ComputeTask class extends RecursiveAction to break the problem into smaller tasks, utilizing work-stealing for efficient load balancing across threads. Additionally, MPI (Message Passing Interface) is utilized for distributed computing across multiple nodes. We employ a robust MPI implementation that facilitates seamless communication between processes, handling data distribution and collection efficiently.

The system is designed for broad compatibility and scalability. JavaFX ensures a smooth and interactive experience on standard desktop environments. This architecture allows for efficient fractal generation and visualization.

6 RESULTS

Results

For depth 5 and a grid size of 243x243, the sequential implementation achieved an average runtime of approximately 13.169 milliseconds. The ForkJoin parallel execution improved this to an average of 11.329 milliseconds. This trend of faster parallel execution was consistent across other problem sizes, with the parallel approach providing substantial performance benefits over the sequential method.

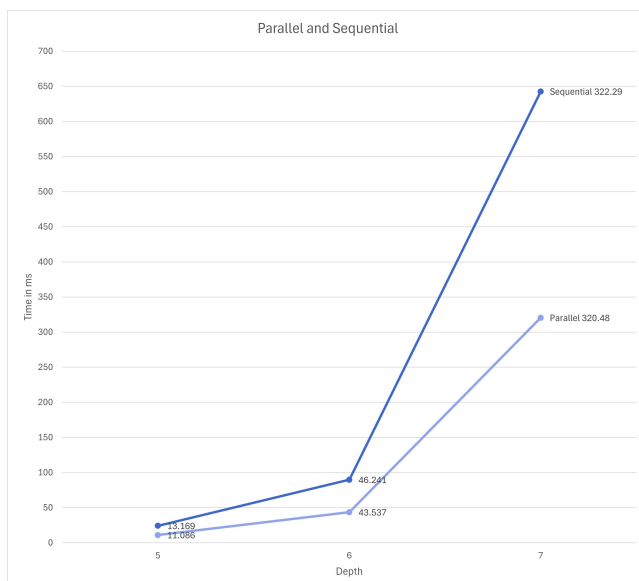

Figure 3: Parallel vs Sequential computation time measurements

Distributed computing using MPI was evaluated with varying numbers of processes. Due to the nature of the grid computation and the requirement for a consistent number of input and output elements, tests were limited to configurations with 3 and 9 MPI

processes. For depth 5 and a grid size of 243x243, 3 MPI processes had an average runtime of 34.143 milliseconds, while 9 processes averaged 36.974 milliseconds. For this grid size and depth, using 3 MPI processes was more efficient than using 9 processes. This result highlights the importance of tuning the number of processes to the problem size and computing environment to achieve optimal performance.

At depth 6 and a grid size of 729x729, the sequential implementation took an average of 46.241 milliseconds, whereas the parallel method achieved an average of 43.537 milliseconds. Distributed computation with 3 MPI processes averaged 466.299 milliseconds, while 9 MPI processes reduced the runtime to 329.394 milliseconds, indicating improved performance with additional processes.

For depth 7 and a grid size of 2187x2187, sequential computation averaged 322.286 milliseconds, and parallel execution was slightly faster at 320.480 milliseconds. Distributed computation showed an average runtime of 10,574.720 milliseconds with 3 processes, which decreased to 6,958.280 milliseconds with 9 processes. These results demonstrate that distributed computing significantly reduces runtime for larger problem sizes, though the benefits of adding more processes may diminish as the problem size grows.

At depth 8 and a grid size of 6561x6561, sequential computation averaged 2,867.27 milliseconds, while parallel execution achieved an average of 2,844.176 milliseconds.

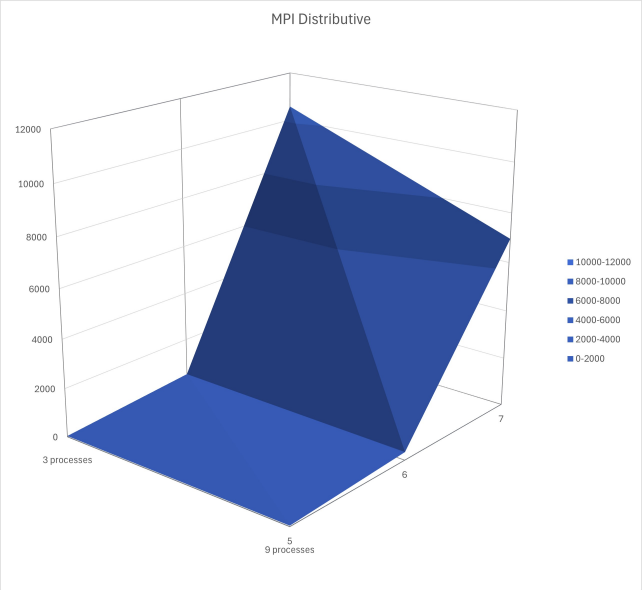

Figure 4: Distributive computation with 3 and 9 processes time measurements

The results indicate that parallel computation using the ForkJoin framework offers better performance than sequential execution across all problem sizes. Distributed computing with MPI is particularly effective for very large grid sizes and deep fractal depths, though the optimal number of processes is constrained by the need to match input and output elements. For your current hardware setup, testing with 3 MPI processes provides a more accurate baseline for performance, avoiding over-subscription and demonstrating how the program performs under typical conditions. Running tests with 9 processes, while not always the most efficient, is valuable for understanding the scalability and upper limits of the application.

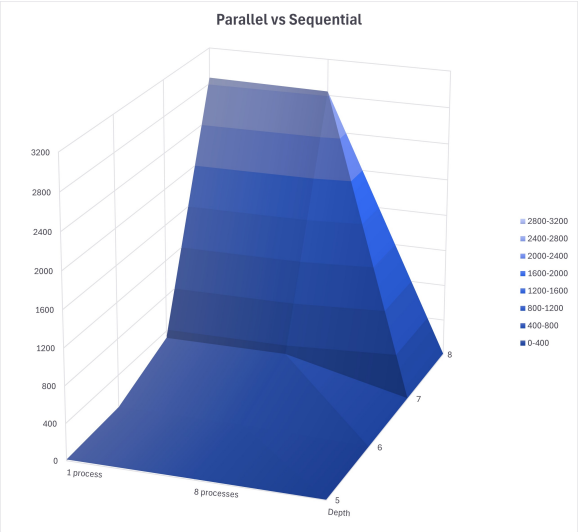

Figure 5: Parallel vs Sequential time measurements 3d representation

7 LINKS

My GitHUb repositories:

https://github.com/janaStani/prog3secondtry

https://github.com/janaStani/prog3project (first try)